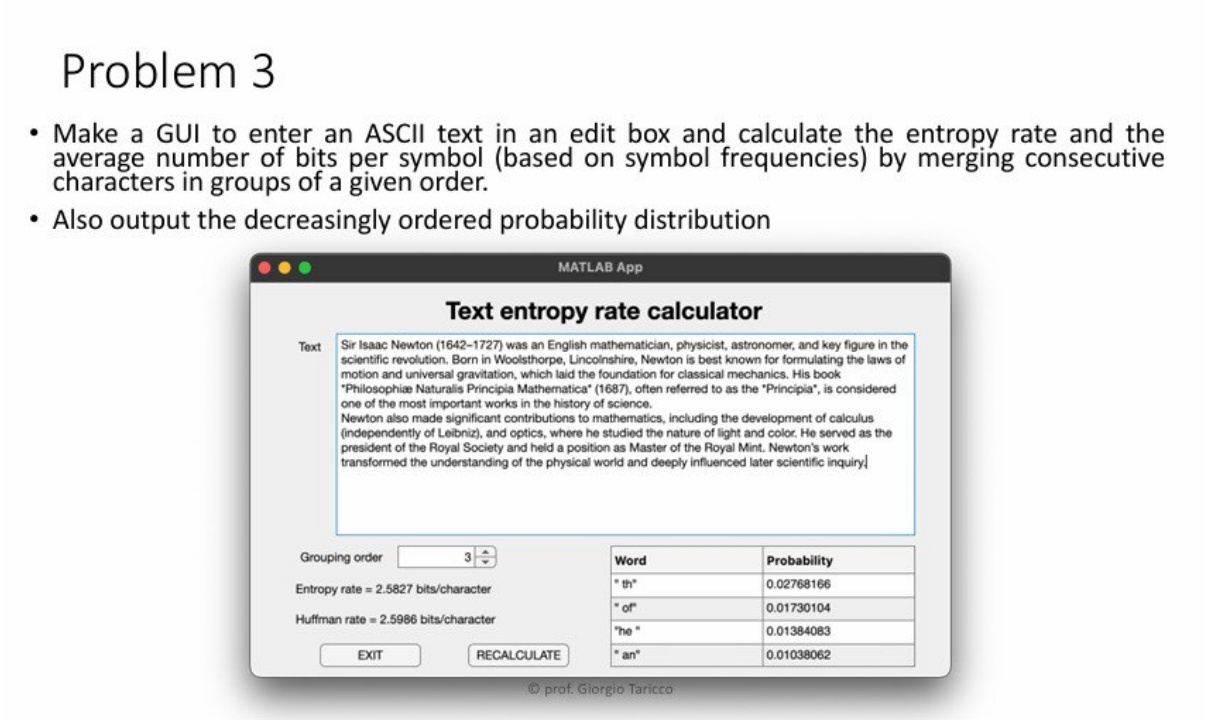
Problem 3 part1: Entropy Rate and Average Bits per Symbol



Introduction

The goal of this task is to calculate the entropy rate and the average number of bits per symbol for a given string of ASCII text. The program analyzes the ASCII values of the characters and groups them based on a specified order size. It then calculates the probability of each symbol and sorts them by their probability.

Entropy Calculation

To calculate the entropy of a set of symbols, we use the following formula:

$$H(X) = -\sum_{i=1}^{N} p(x_i) \log_2 p(x_i)$$

In this case, the script calculates entropy for *n-grams* (sequences of n consecutive characters), where the symbols of the sequence are treated as unique entities. We can calculate the entropy of n-grams (where n is the grouping size) using the formula:

$$H(X_n) = -\sum_{i=1}^{M} p(x_i) \log_2 p(x_i)$$

where M is the number of unique n-grams in the text.

Average Bits per Symbol

By calculating the average number of bits per symbol, we determine how efficiently we can encode the data. This is given by the formula:

$$B_{\text{avg}} = \sum_{i=1}^{M} p(x_i) \log_2 \frac{1}{p(x_i)}$$

Grouping and ASCII Encoding

This part is key to how the program works. The text is transformed into numbers that we can analyze, which is done through ASCII encoding and then grouping characters into n-grams.

ASCII Encoding

The input text is converted into ASCII values. Each character is replaced by its corresponding ASCII number. This step allows us to analyze how often certain characters (or groups of characters) appear in the text, as numbers are easier to work with than letters.

Grouping into n-grams

Once the text is converted into ASCII values, the characters are grouped into chunks called *n-grams*. An n-gram is a sequence of n consecutive characters. The code groups the input text into n-grams based on the specified number n. After grouping, each n-gram is converted into a unique number by summing the ASCII values of its characters, with each value being weighted by powers of 256. This is because each ASCII character fits in a byte, and a byte can represent 256 different values.

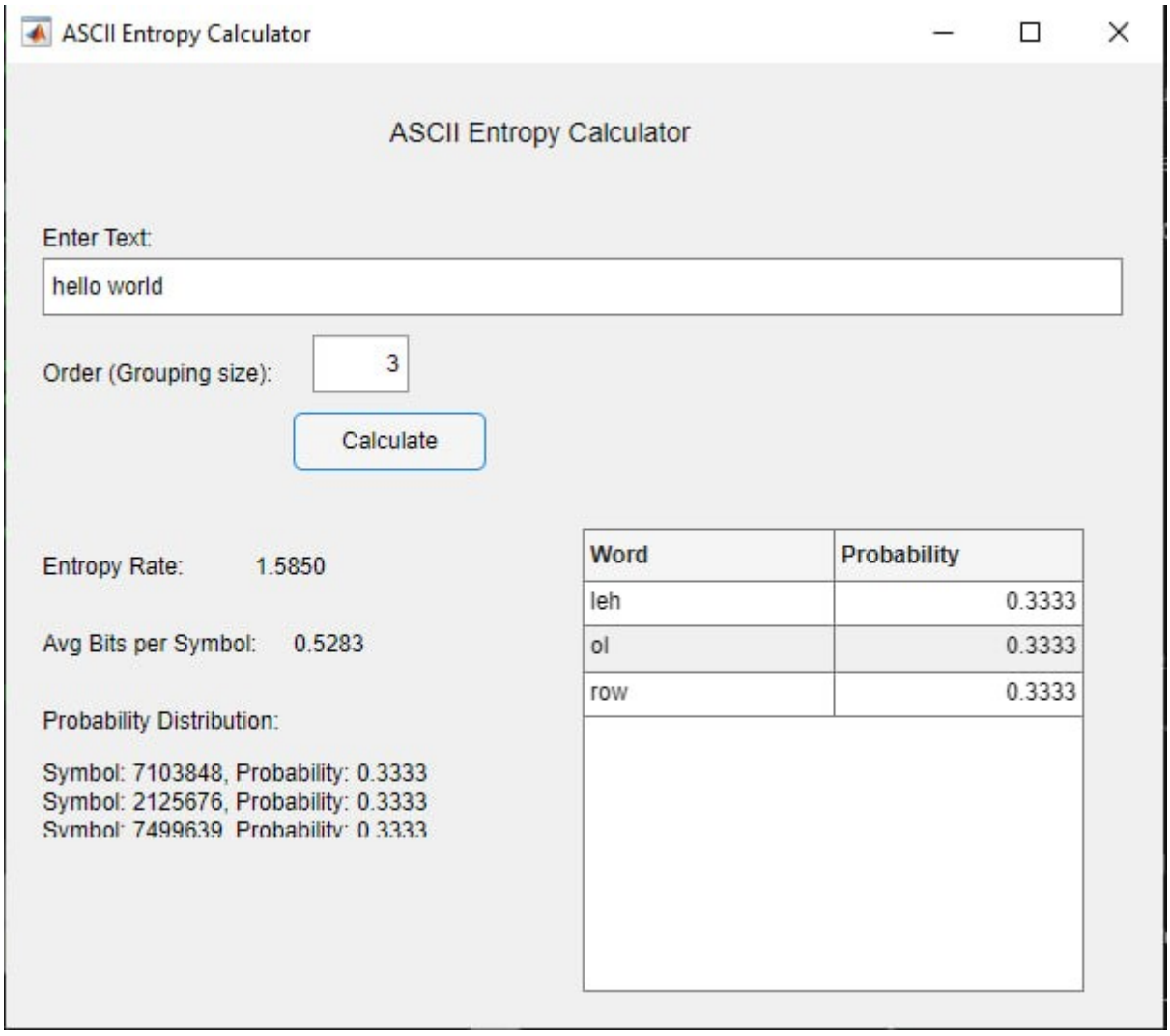
Calculating Probability Distribution

After creating the n-grams, the script calculates the frequency of each unique n-gram using the MATLAB function `unique()`. The result is an array of unique symbols (n-grams), and the counts of their occurrences are used to calculate their probabilities:

$$p_x = \frac{\text{Frequency of } x_i}{\text{Total number of n-grams}}$$

The n-grams are then sorted in decreasing order of probability, with the most frequent n-grams coming first. This sorting process is done using the MATLAB function `sort()`, which arranges the probabilities and n-grams in descending order.

Conclusion



The results are displayed in the GUI, with the probability distribution shown as text and also as a table in the MATLAB GUI, where each n-gram and its corresponding probability are listed.